Proyecto multijugador

Motivación

En este documento se muestra una forma sencilla para implementar la opción multijugador en el proyecto, tal y como se requiere en el enunciado. La propuesta, si bien no es la más completa, es funcional y se podrá mejorar en futuras iteraciones.

Ciclo de vida del proyecto

El proyecto, en su etapa actual, sigue un bucle que se itera mientras no se haya marcado el juego como finalizado. En `game_loop.c` se encuentra la función encargada de ir actualizando la interfaz del juego, recogiendo la entrada del usuario y actuando en base a ésta hasta que se termine el juego.

Hasta ahora este ciclo se repetía una y otra vez, y era siempre el mismo jugador el que interactuaba. Ahora, para posibilitar el multijugador, habrá que tener en consideración turnos. Para simplificar, los turnos siempre serán rotatorios entre los jugadores: cada uno podrá ejecutar uno y solo un comando antes de pasar el control al siguiente jugador. Además, de nuevo para simplificar, si un jugador muere, se dará por finalizado el juego.

Cambios en el modelo de datos

Para dar cabida al multijugador, habrá que hacer, primero, unos cambios en la estructura de `Game`. Lo primero será que en lugar de tener un único puntero a `Player`, tendremos una lista de punteros a `Player`. Esta lista, al igual que los espacios, objetos, etc. será estática, definida mediante la macro `MAX_PLAYERS` que estableceremos en 8 (ya veremos por qué).

Una vez tengamos la lista de jugadores, tendremos que hacer algunos cambios más en la estructura para que la información que se presente tenga sentido. Ahora mismo, la estructura de `Game` cuenta con campos que nos permiten representar información del juego en pantalla, como son `last_command`, `last_status` o `message`. Ahora estos campos deberían poder ser particularizados para cada jugador. Una forma sencilla de resolverlo es declarar una nueva estructura `InterfaceData`, por ejemplo, que contenga estos y cualquier otro campo que contenga información de la interfaz propia de cada jugador.

Una vez declarado este nuevo tipo de dato, en la estructura de `Game` borraremos los campos de información de interfaz y añadiremos un array de punteros a esta estructura (array estático, de tamaño `MAX_PLAYERS`). Cada elemento de este array contendrá la información de interfaz de cada jugador.

Finalmente, el juego deberá contar con un nuevo campo que gestione los turnos (`turn`). Este valor podremos gestionarlo en dos sentidos: o bien esta variable nos indica el jugador al que le toca (su posición en el array) o bien es un número incremental que sube en cada cambio de jugador. Esta decisión tendrá implicaciones, un poco más adelante, en cuanto a la lógica de `Game`. Independientemente de lo que signifique el valor del turno, tendremos que implementar las funciones para poder recuperarlo y cambiarlo (`game_get_turn` y `game_next_turn` respectivamente).

Cambios en la lógica del juego

De cara a simplificar y minimizar los cambios en cuanto a `Game` y sus funciones, vamos a considerar que todas las funciones de `Game` que están relacionadas con el jugador o la interfaz se van a referir al jugador que tiene el turno en el momento de ser llamadas. Es decir, por ejemplo, la función `Player *game_get_player(Game *game)` devuelve el puntero al jugador que tiene el turno. De este modo, no tendremos que cambiar el prototipo de las funciones de `Game`, salvo una: `game_set_player_location`. Esta función la usamos cuando creamos a los jugadores y, por lo tanto, la tendremos que actualizar para que reciba el `Id` del jugador.

En el resto de funciones, como hemos dicho, trabajaremos con el jugador que tiene el turno pero, ¿cómo? Aprovechándonos del nuevo campo `turn`. Aquí surgen dos variantes:

1. Si hemos decidido que este campo indica el jugador (su posición en el array de jugadores), entonces es muy fácil, ya que basta con actualizar las llamadas a `game->player` por `game->player[game->turn]`.
2. Si hemos decidido que el turno es un número incremental, las llamadas serán similares, pero tendremos que hacer el módulo del número de jugadores. Es decir, en cada llamada a `game->player` tendríamos que hacer `game->player[game->turn%game->n_players]`.

En una opción añadimos la complejidad a la hora de calcular el turno (opción 1) y en la otra simplificamos ese cálculo, pero añadimos algunas restricciones en otras partes del programa.

Cambios en la representación gráfica

Tal y como está organizado y modularizado el programa, la representación gráfica debería adaptarse al jugador activo en cada momento de forma casi automática. Es decir, al centralizar en `Game` toda la información del juego y usar la interfaz que nos ofrece para recuperar la información, como no se ha variado la interfaz de éste, el sistema carga todo lo necesario independientemente del número de jugadores.

Sin embargo, en lo que respecta a la biblioteca `libscreen` sí hay un pequeño cambio. Esta biblioteca se encarga de la gestión a bajo nivel de la interfaz: tamaños de las secciones de la interfaz, colores, etc. De cara a poder dar feedback visual de los cambios de jugador (cambiar el color del marco de la interfaz, hasta ahora azul), hemos generado una nueva biblioteca `libscreen` que podéis descargar de Moodle. En concreto, el cambio clave está en la función `void screen_paint (FrameColor color)`. Esta función hasta ahora no recibía ningún argumento, y ahora recibe un `FrameColor` (que no es más que un tipo enumerado que representa los colores).

En concreto este tipo enumerado está definido del siguiente modo:

```
typedef enum {BLUE, GREEN, BLACK, RED, YELLOW, PURPLE, CYAN, WHITE} FrameColor;
```

De esta forma, la propia biblioteca asigna un color al marco en función del valor recibido. De ahí que el número máximo de jugadores sea 8, ya que podemos pintar en 8 colores diferentes.

Otro punto a modificar en la interfaz y que ayudaría a identificar al usuario que tiene el turno (sobre todo para depurar) es actualizar el texto que aparece en la zona llamada banner, que actualmente contiene el texto “The anthill game”.

Preguntas frecuentes

¿Puedo seguir con el juego aunque muera uno de los jugadores? Para simplificar, os proponemos que el juego termine cuando muera cualquiera de los jugadores. Sin embargo, en la I4 sí puede ser una funcionalidad útil dependiendo de vuestro juego.

¿Puedo jugar varios turnos (por ejemplo, seguir atacando)? De nuevo, para simplificar la implementación, os proponemos que cada jugador juegue un único comando. Es más, el jugador ejecutará el comando y no verá el resultado (por ejemplo, si ha hecho un ataque, el cambio en la vida del adversario) hasta que le vuelva a tocar.

¿Puedo cambiar los colores? De momento, mejor no. Cambiar los colores supone cambiar la biblioteca `libscreen` y puede ser algo complejo. Lo hemos implementado de forma que no haya interacción entre `libscreen` y otros módulos distintos a `graphic_engine`, que es el único que gestiona la interfaz.

Al ejecutar el comando, se cambia automáticamente de jugador y, hasta que no llega el turno de nuevo, no veo el resultado (por ejemplo, el resultado de un attack). ¿Se puede solucionar esto? Sí, claro que se puede. Básicamente, en el bucle del juego, se pinta la interfaz, se recoge la entrada del usuario, se ejecuta la acción y, por último, se incrementa el turno. En este bucle podemos repintar la interfaz antes de cambiar de turno. Además, para que dé tiempo a ver el resultado, se puede incluir un pequeño retardo antes de incrementar el turno y terminar el bucle. Para ello, puedes usar el siguiente código:

```
#include <unistd.h>
...
/* Espera durante 3s */
sleep(3);
...
```

Este retardo puedes ejecutarlo por ejemplo, si hay 2 o más jugadores, de forma que en el caso de un único jugador el juego sea más fluido, sin retrasos obligados.

¿Cómo puedo gestionar el fichero de log con múltiples usuarios? Lo más sencillo en este sentido es considerar que cada entrada del fichero de log corresponde con un jugador, y estos se alternan. Si queréis profundizar un poco más, podríais plantear que se indicase el número de jugador al principio de cada línea, o algún mecanismo similar. Siempre, sin romper las normas de modularidad y opacidad que marcamos en la iteración.

Lo mismo puede ocurrir con los ficheros de entrada para las pruebas de integración. Lo más sencillo y directo es prepararlos de forma que los jugadores roten uno tras otro, pero se podría plantear algún formato más sofisticado y que fuese lo suficientemente flexible para permitir que un jugador ejecutase varios comandos (si lo tuviéseis así implementado)